

Sistemas Robótico Moviles

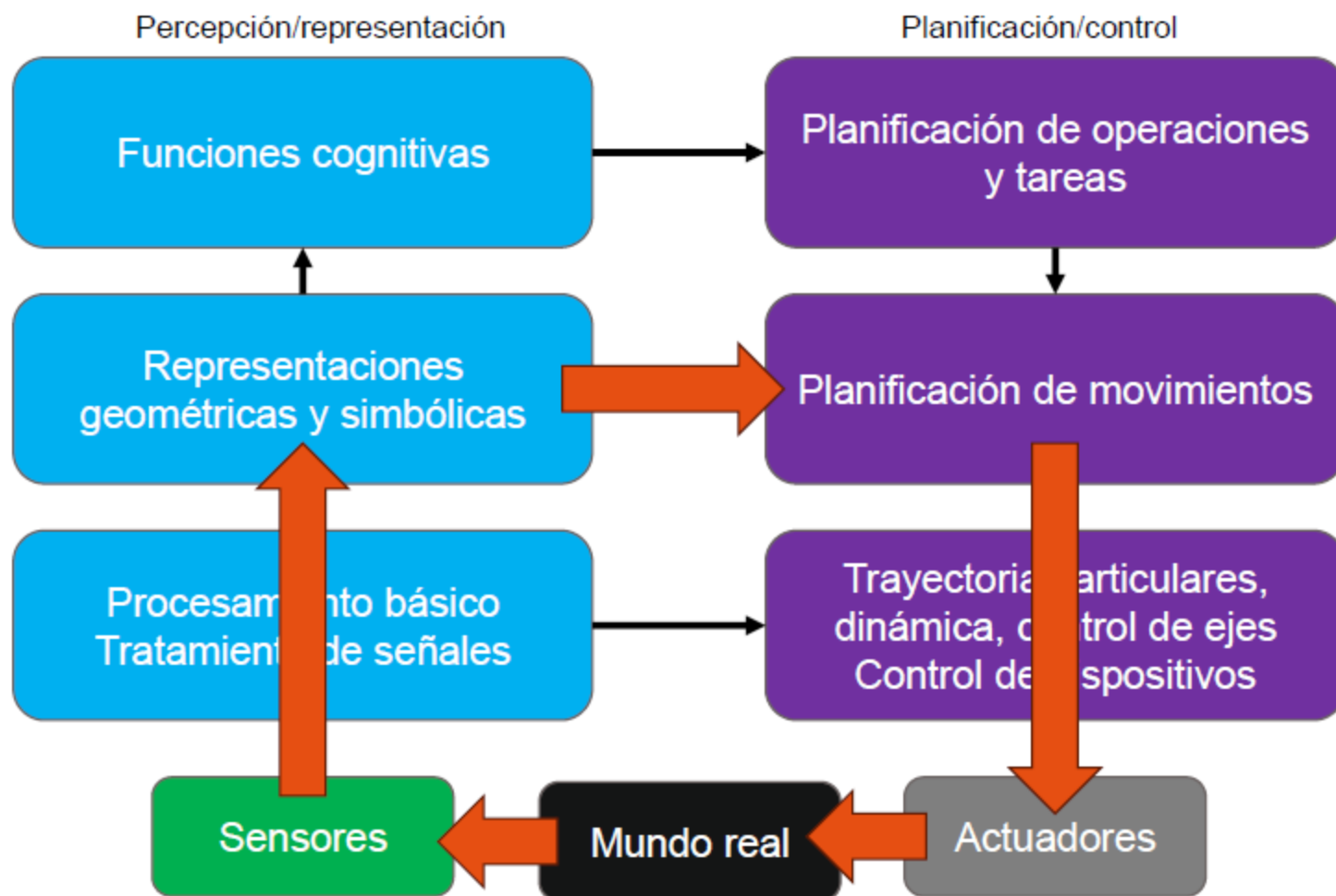


Universidad
Internacional
de Valencia

Tema 3: Control y navegación en robótica móvil

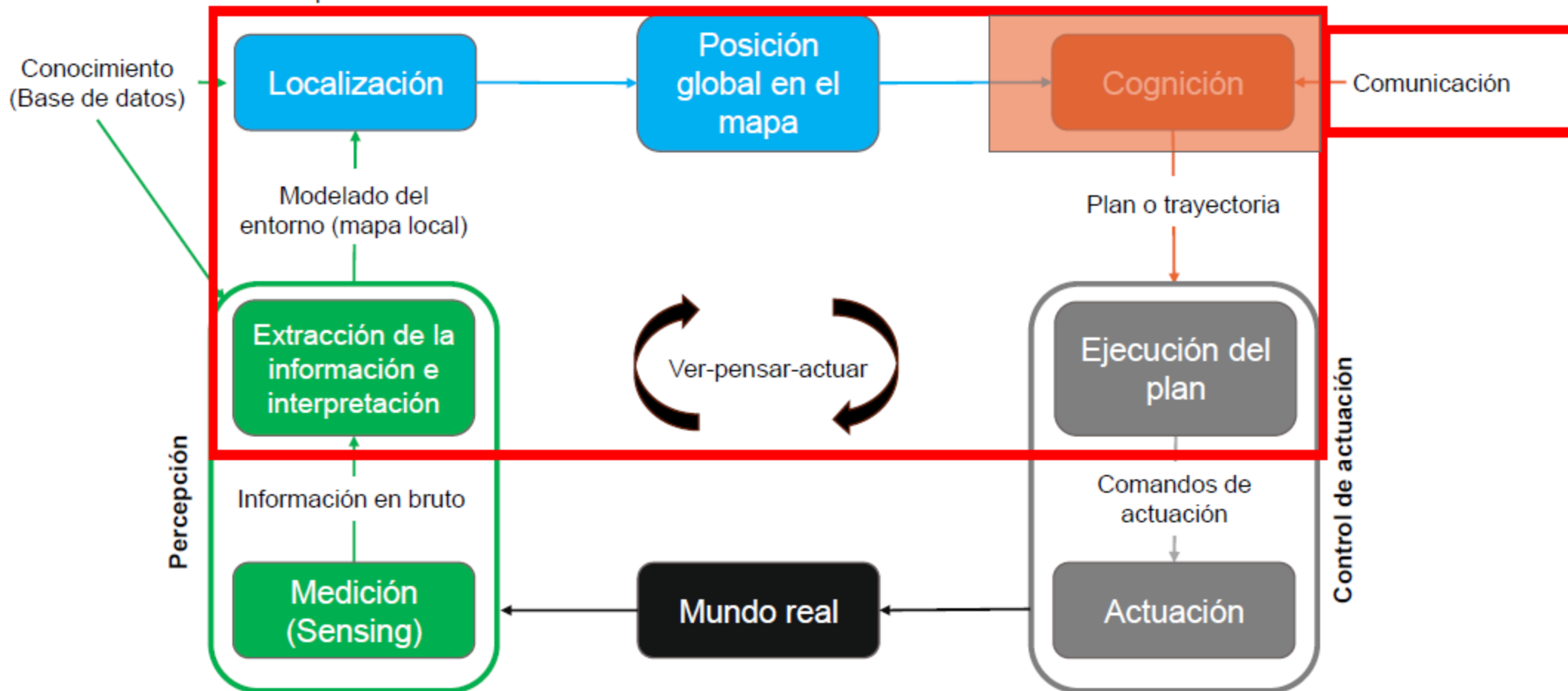
Fundamentos del diseño y arquitectura: diseño funcional

El enfoque típico es partir las tareas que son necesarias de realizar y descomponerlas en funciones, definiendo las interacciones entre ellas.



Fundamentos del diseño y arquitectura

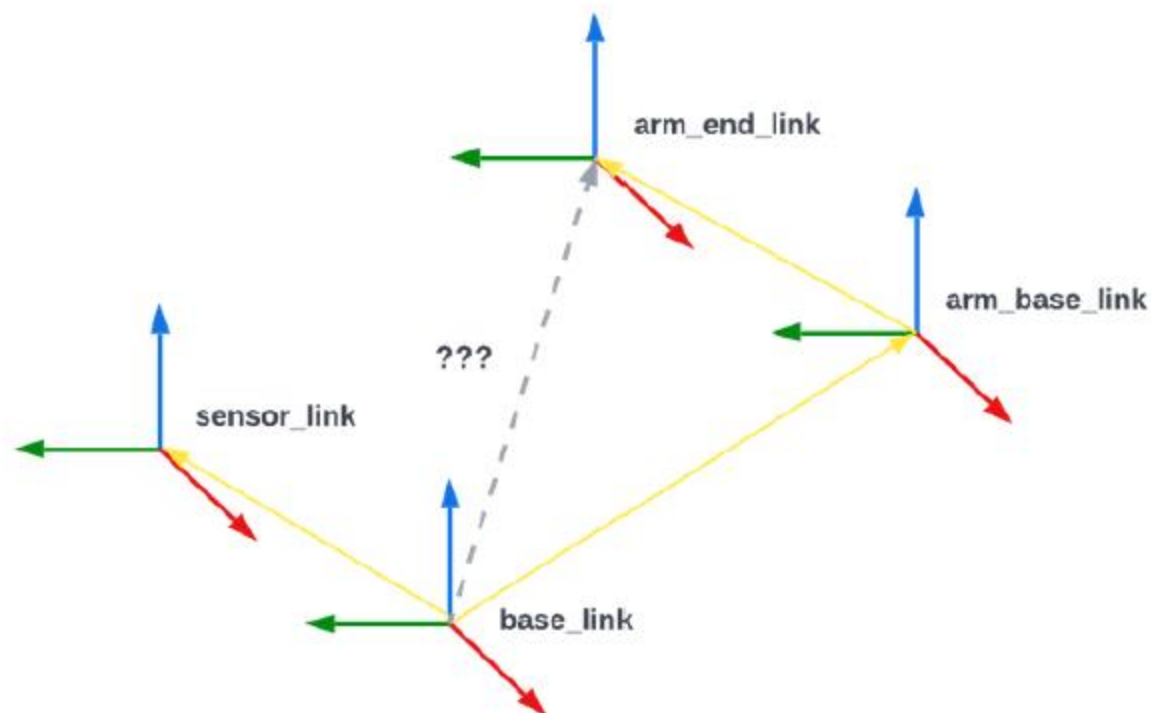
Esquema de referencia de control para robots móviles



Transformaciones Relativas

Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

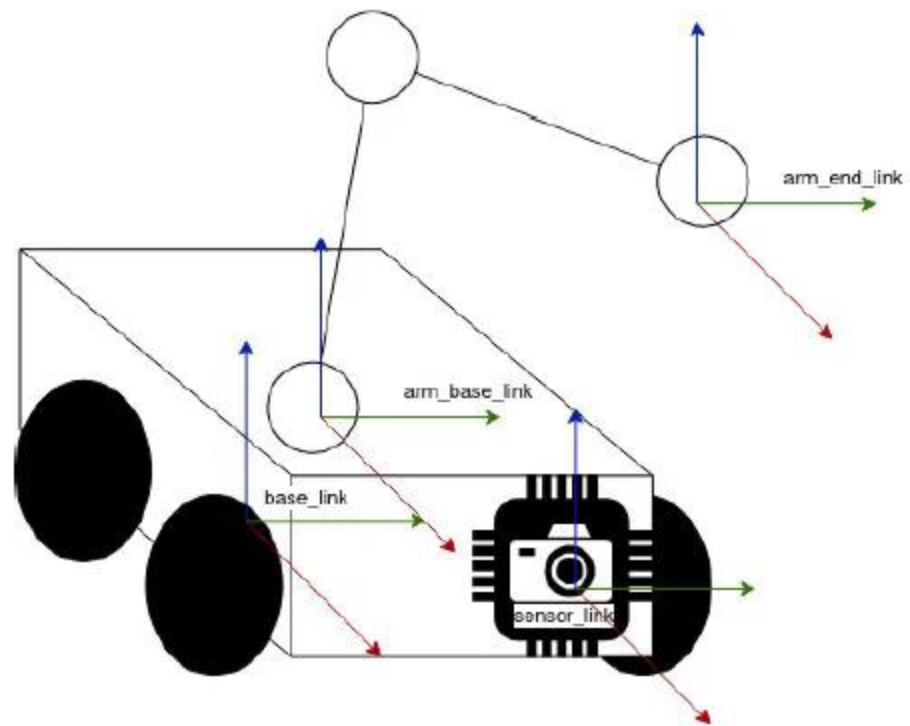
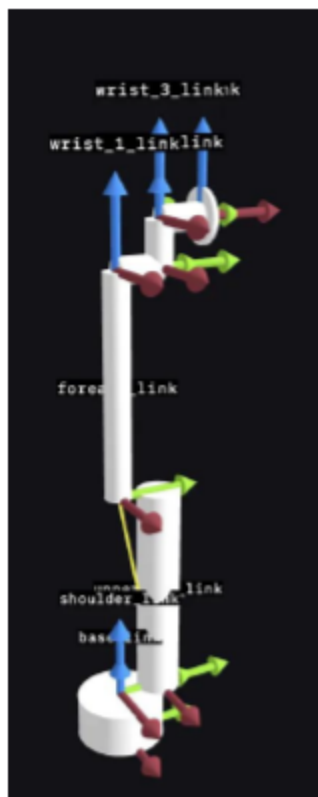
Estas transformaciones son fundamentales para entender como las diferentes partes de un robot se relacionan entre sí y como interactúa con su entorno.



Transformaciones Relativas

Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

Estas transformaciones son fundamentales para entender como las diferentes partes de un robot se relacionan entre sí y como interactúa con su entorno.

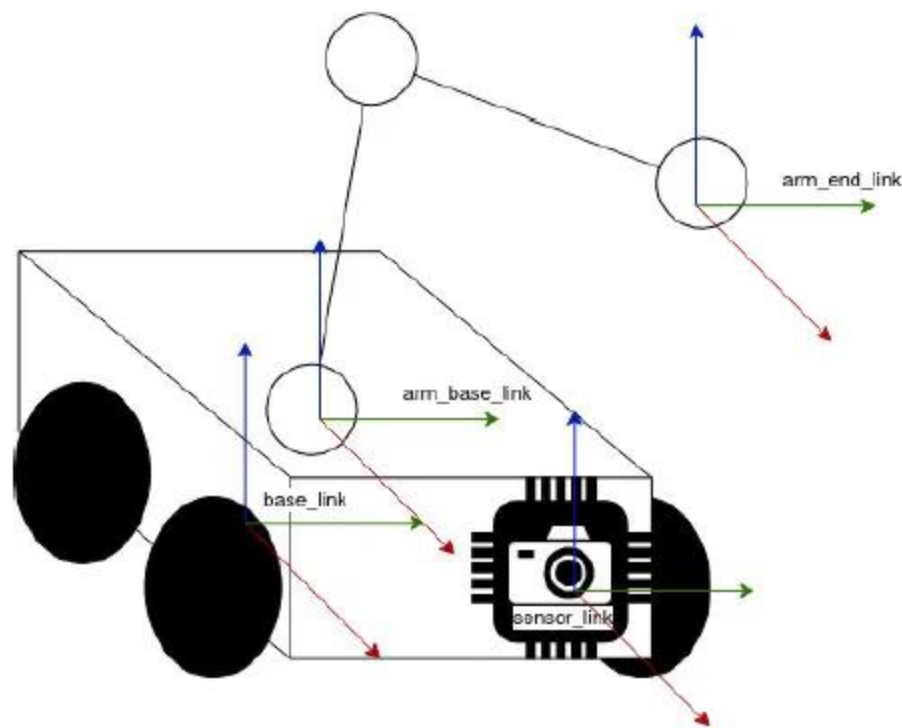


Transformaciones Relativas

Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

Los objetos se sitúan en **frames** (marcos o ventanas de coordenadas) que describen la posición y orientación de un objeto, típicamente en los ejes x,y,z. Cada frame emplea su propio identificador ("frame_id").

- Toda escena debe tener una **base_frame**, que normalmente suele ser el mundo (**world**) o el mapa (**map**)
- El **base_link** del robot normalmente **se sitúa en su base**, y sirve para definir frames hijos para los sensores y actuadores.
- Un frame hijo, puede tener a su vez más frames hijos (véase un brazo robótico)
- Las transformaciones definen las rotaciones y traslaciones de un frame a otro.
 - Podemos transformar:
 - Padre a hijo
 - Hijo a padre
 - Transformar entre diferentes generaciones de frames
 - Estas transformadas definen el árbol de transformadas (**transformation tree**)
 - Las transformadas pueden ser estáticas (tf_static) o dinámicas (tf)



Transformaciones Relativas

Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

Los objetos se sitúan en **frames** (marcos o ventanas de coordenadas) que describen la posición y orientación de un objeto, típicamente en los ejes x,y,z. Cada frame emplea su propio identificador ("frame_id").

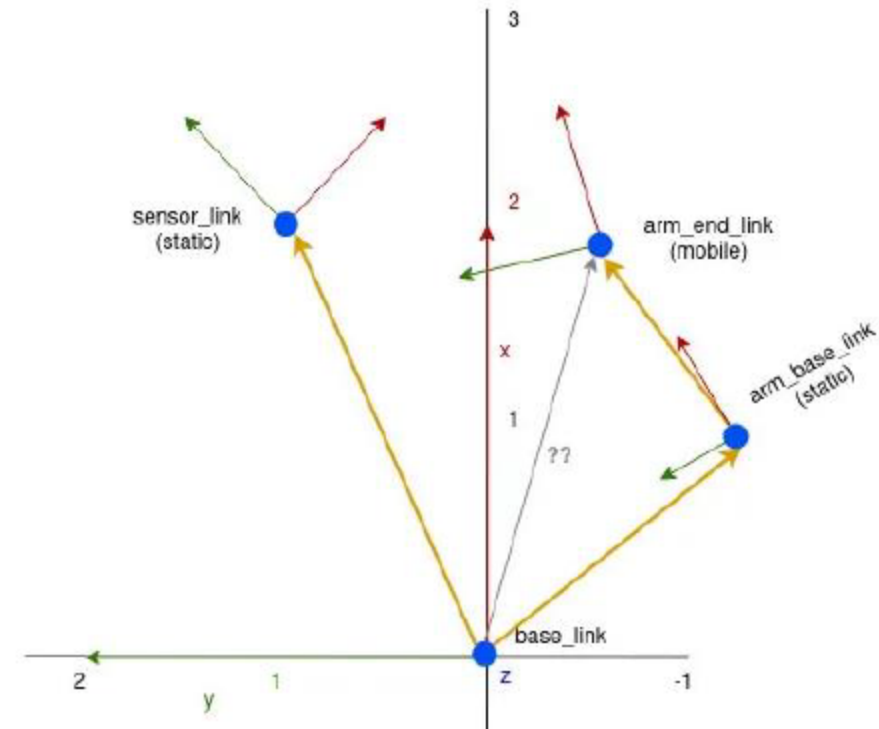
Creando un árbol de transformadas simple:

Padre: base_link

sensor_link	posición (+2,+1), rotación -45°
arm_base_link	posición (+1,-1), rotación $+30^\circ$

Padre: arm_base_link

arm_end_link	posición (+1,+1), sin rotación
--------------	--------------------------------



Transformaciones Relativas

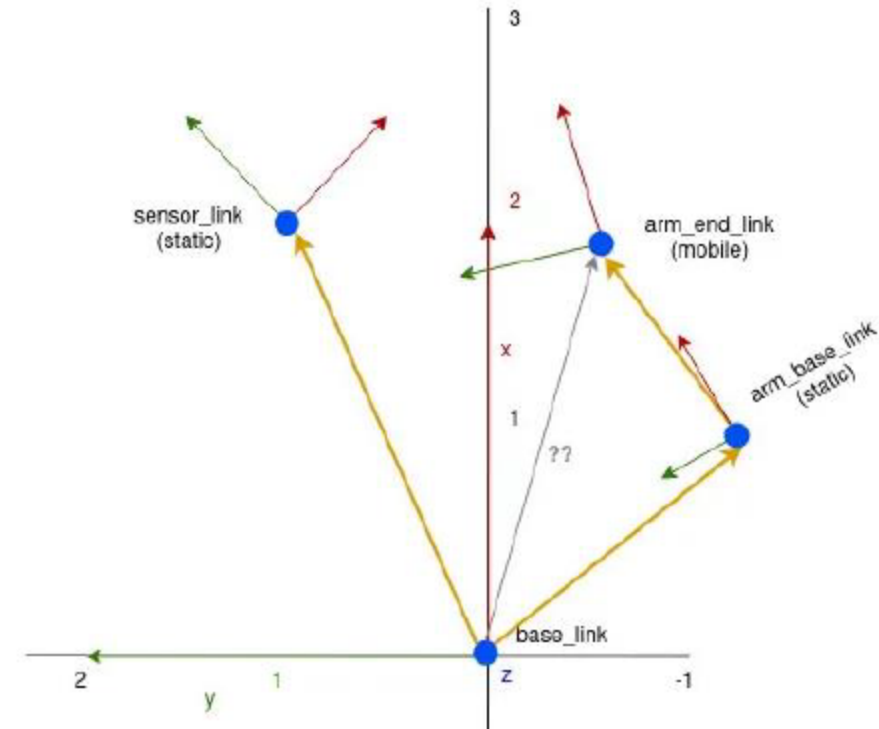
Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

Los objetos se sitúan en **frames** (marcos o ventanas de coordenadas) que describen la posición y orientación de un objeto, típicamente en los ejes x,y,z. Cada frame emplea su propio identificador ("frame_id").

Calculando la transformada entre base_link y arm_end_base_link

- T – transformada bidimensional
- P – Vector de posición

$$T = \begin{bmatrix} \cos(\phi) & -\sin(\phi) & d_x \\ \sin(\phi) & \cos(\phi) & d_y \\ 0 & 0 & 1 \end{bmatrix} \quad \vec{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Transformaciones Relativas

Las **transformaciones relativas** definen las relaciones entre posiciones y orientaciones del robot, sus partes y el mundo que lo rodea.

Los objetos se sitúan en **frames** (marcos o ventanas de coordenadas) que describen la posición y orientación de un objeto, típicamente en los ejes x,y,z. Cada frame emplea su propio identificador ("frame_id").

Calculando la transformada entre base_link y arm_end_base_link

- T – transformada bidimensional
- P – Vector de posición

Padre: base_link

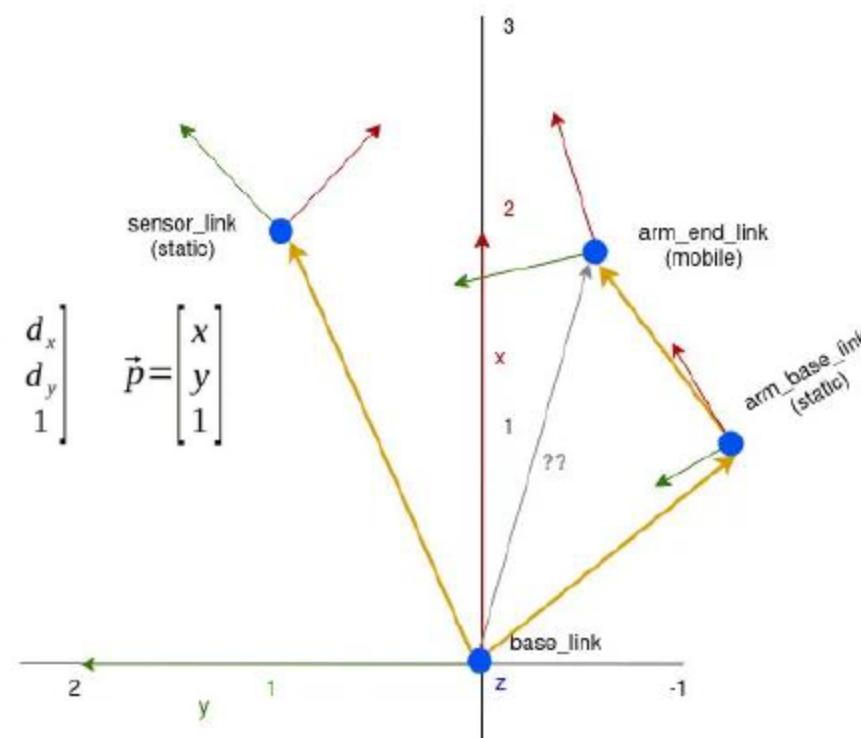
sensor_link posición (+2,+1), rotación -45° (pi/4)
 arm_base_link posición (+1,-1), rotación +30° (pi/6)

$$T = \begin{bmatrix} \cos(\phi) & -\sin(\phi) & d_x \\ \sin(\phi) & \cos(\phi) & d_y \\ 0 & 0 & 1 \end{bmatrix} \quad \vec{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Padre: arm_base_link

arm_end_link posición (+1,+1), sin rotación

$$\vec{p}_{base}^{armend} = T_{base}^{armbase} \cdot \vec{p}_{armbase}^{armend} = \begin{bmatrix} \cos(\frac{\pi}{6}) & -\sin(\frac{\pi}{6}) & 1 \\ \sin(\frac{\pi}{6}) & \cos(\frac{\pi}{6}) & -1 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\frac{\pi}{6}) - \sin(\frac{\pi}{6}) - 1 \\ \sin(\frac{\pi}{6}) + \cos(\frac{\pi}{6}) - 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.366 \\ 0.366 \\ 1 \end{bmatrix}$$



Construcción del mapa

Para crear un mapa:

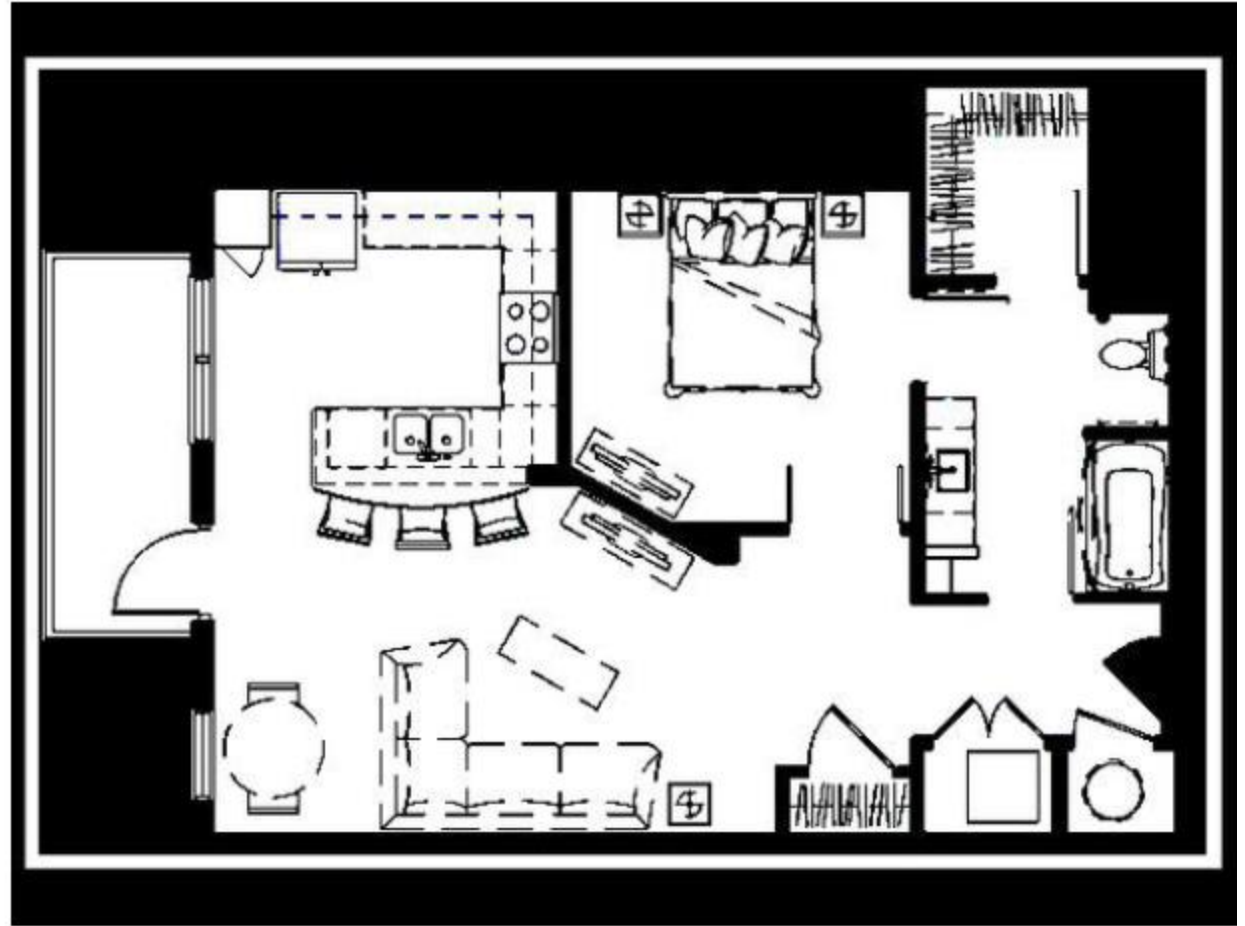
1. Conseguir un mapa de tu planta para el robot.



Construcción del mapa

Para crear un mapa:

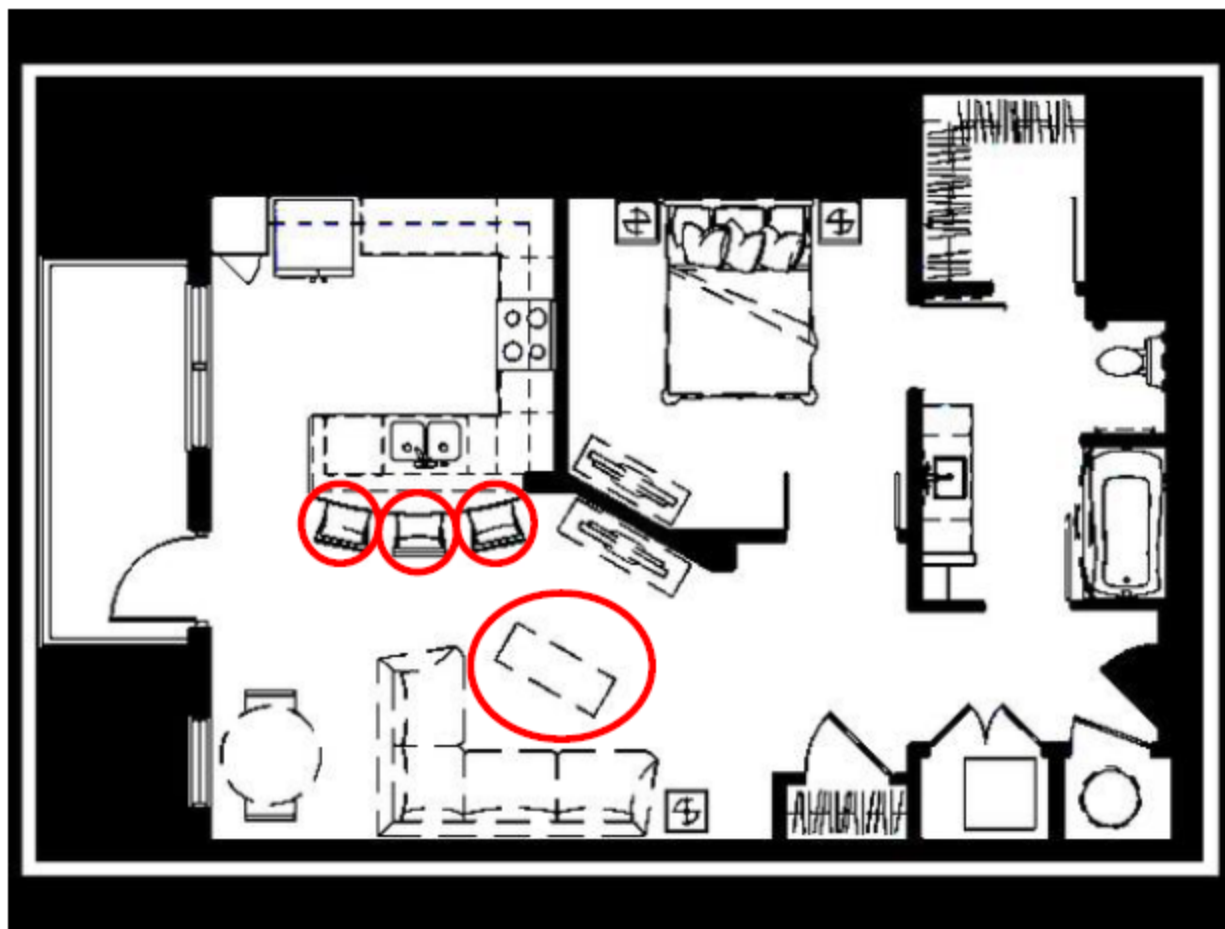
1. Conseguir un mapa de tu planta para el robot.
2. Binarizarlo (convertirlo a escala de blanco-negro)
Negro = obstáculo
Blanco = libre para el paso del robot



Construcción del mapa

Para crear un mapa:

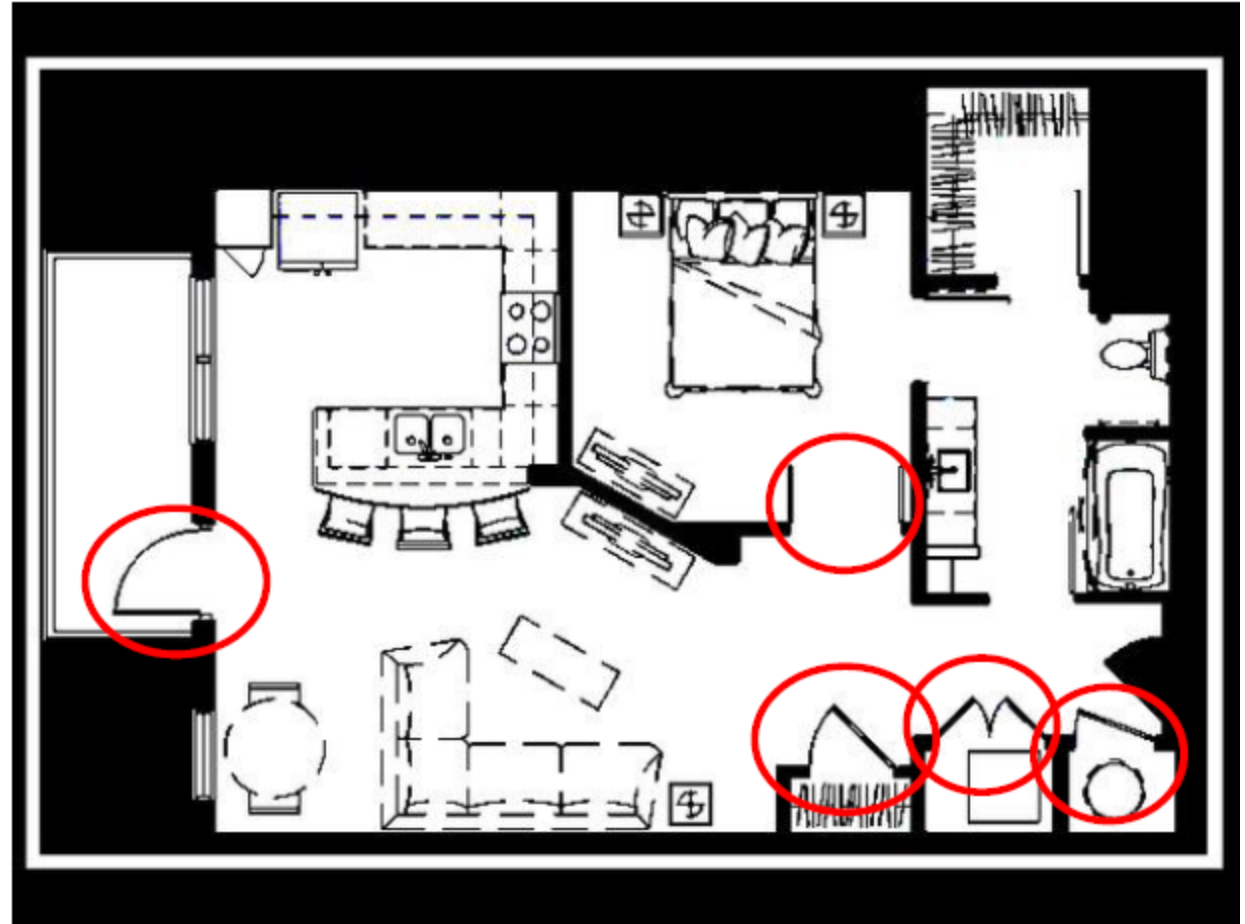
1. Conseguir un mapa de tu planta para el robot.
2. Binarizarlo (convertirlo a escala de blanco-negro)
Negro = obstáculo
Blanco = libre para el paso del robot
3. Eliminar objetos móviles



Construcción del mapa

Para crear un mapa:

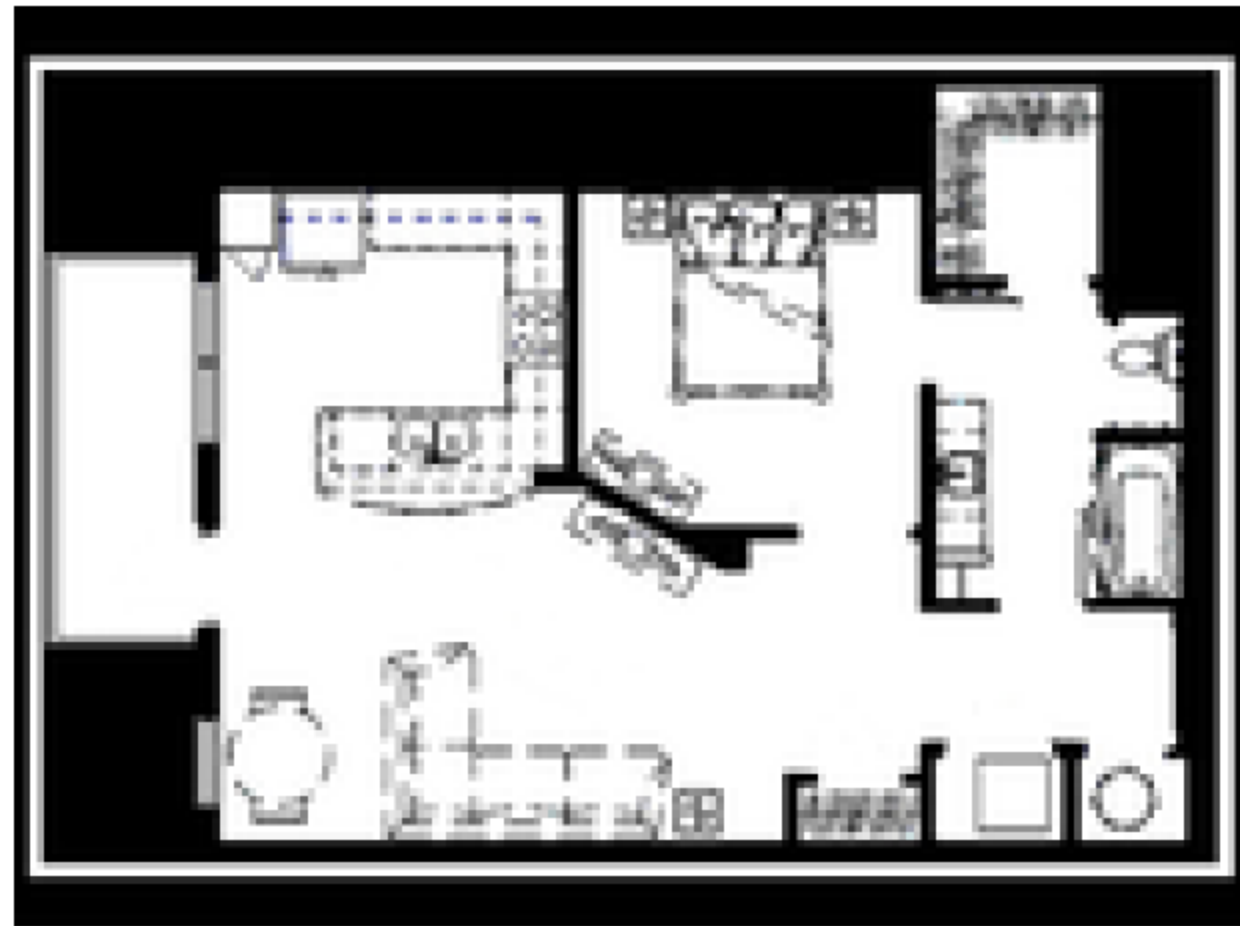
1. Conseguir un mapa de tu planta para el robot.
2. Binarizarlo (convertirlo a escala de blanco-negro)
Negro = obstáculo
Blanco = libre para el paso del robot
3. Eliminar objetos móviles
4. Eliminar todos los objetos móviles



Construcción del mapa

Para crear un mapa:

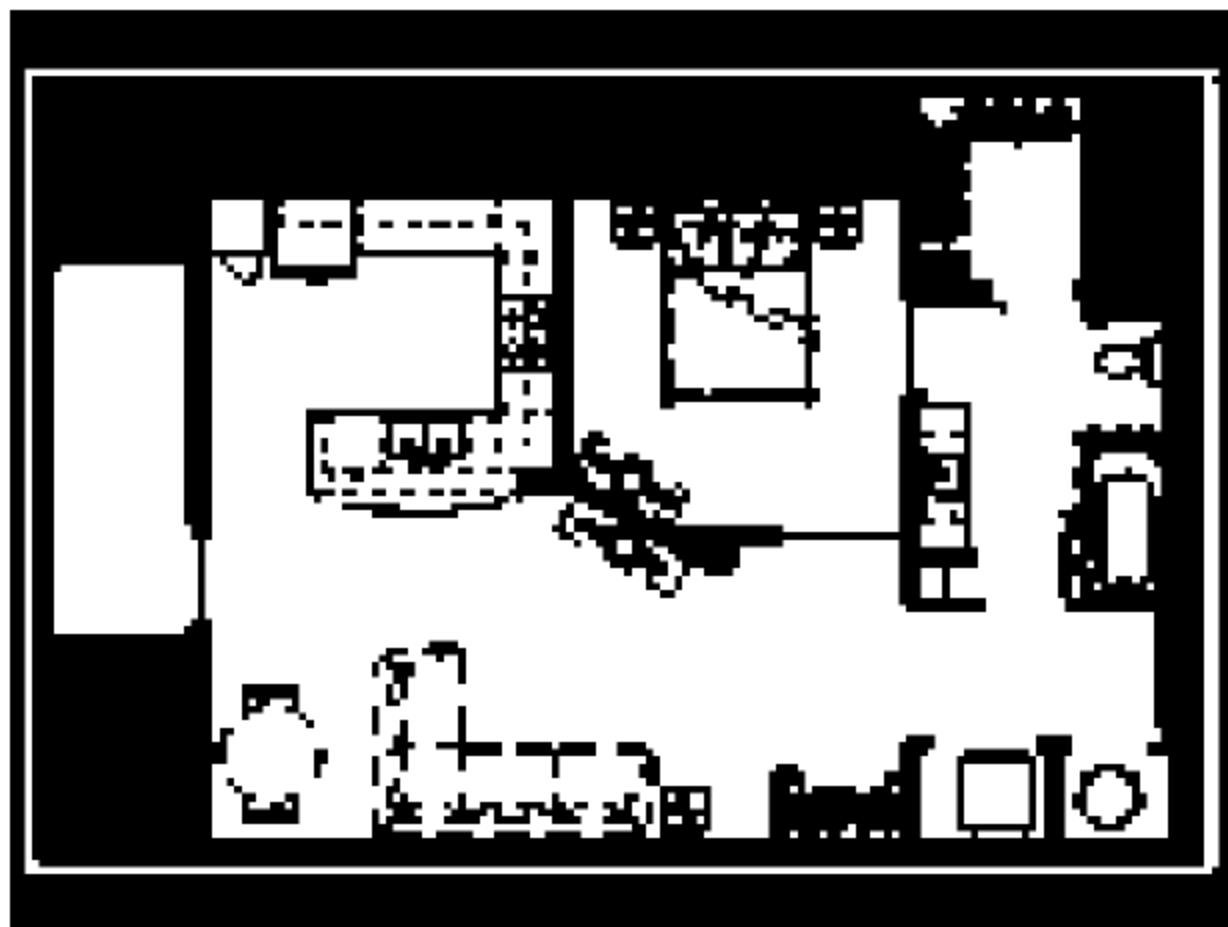
1. Conseguir un mapa de tu planta para el robot.
2. Binarizarlo (convertirlo a escala de blanco-negro)
Negro = obstáculo
Blanco = libre para el paso del robot
3. Eliminar objetos móviles
4. Eliminar todos los objetos móviles
5. Definir este mapa como **mapa de localización**
6. Guardar el ratio px/m (va a hacer falta para el robot)
Resoluciones típicas: (0,05-0,01)m/px (50cm-10cm)
7. Hacer un downsampling del mapa para navegación (ayudará a salvar potencia de computo)



Construcción del mapa

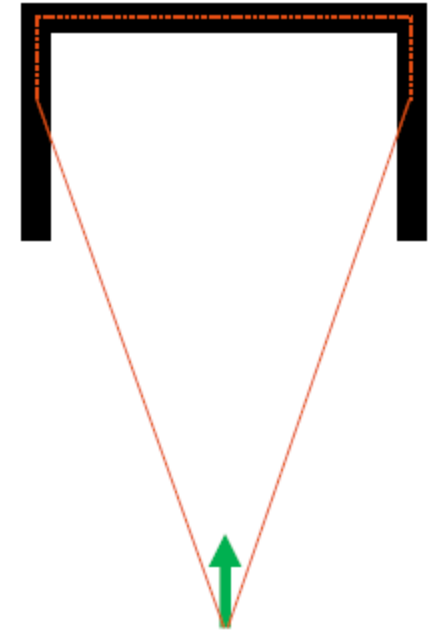
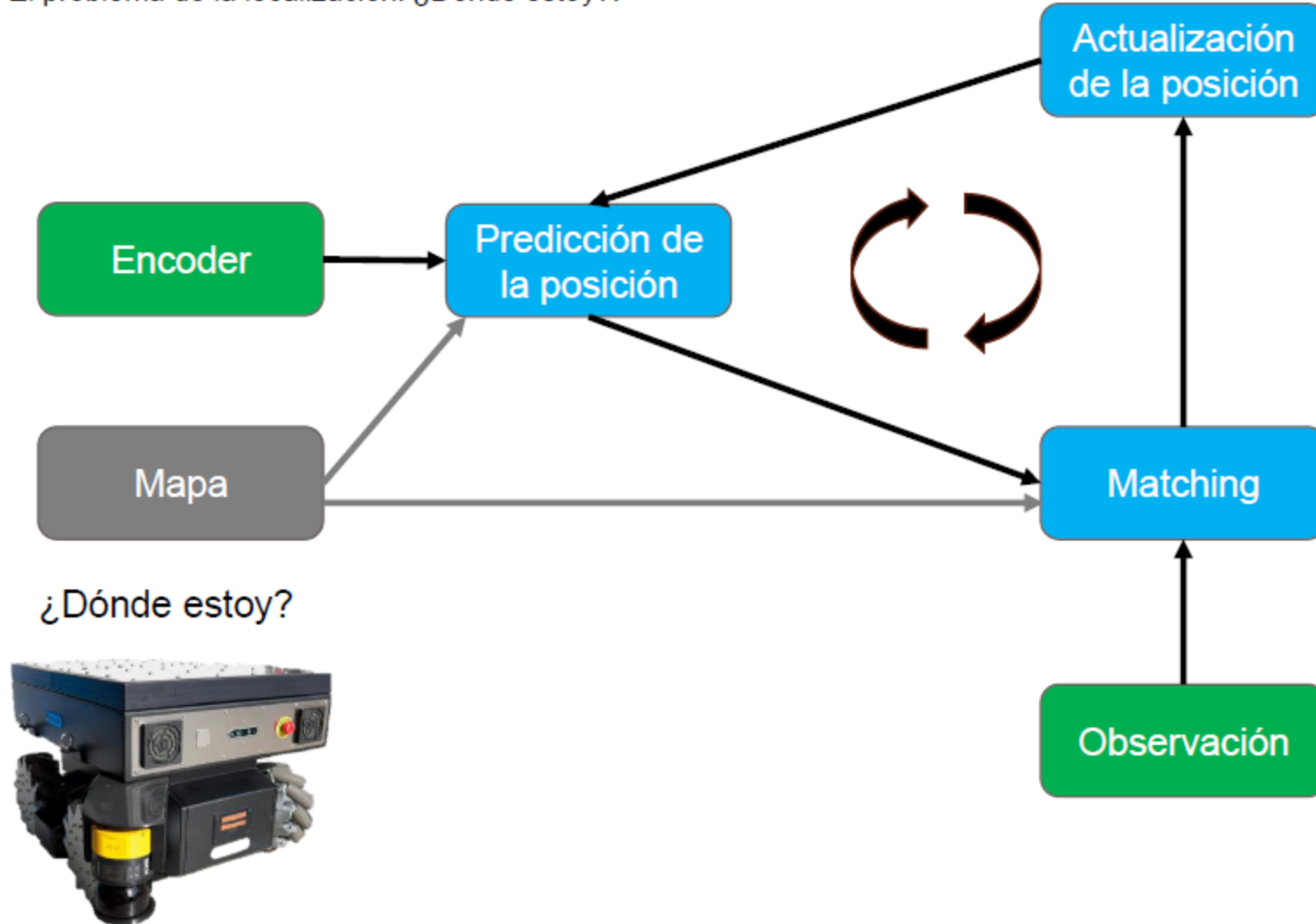
Para crear un mapa:

1. Conseguir un mapa de tu planta para el robot.
2. Binarizarlo (convertirlo a escala de blanco-negro)
Negro = obstáculo
Blanco = libre para el paso del robot
3. Eliminar objetos móviles
4. Eliminar todos los objetos móviles
5. Definir este mapa como **mapa de localización**
6. Guardar el ratio px/m (va a hacer falta para el robot)
Resoluciones típicas: (0,05-0,01)m/px (50cm-10cm)
7. Hacer un downsampling del mapa para navegación (ayudará a salvar potencia de computo)
8. Volver a umbralizar
9. Bloquear zonas de paso al robot
10. Guardar mapa como **mapa de navegación**



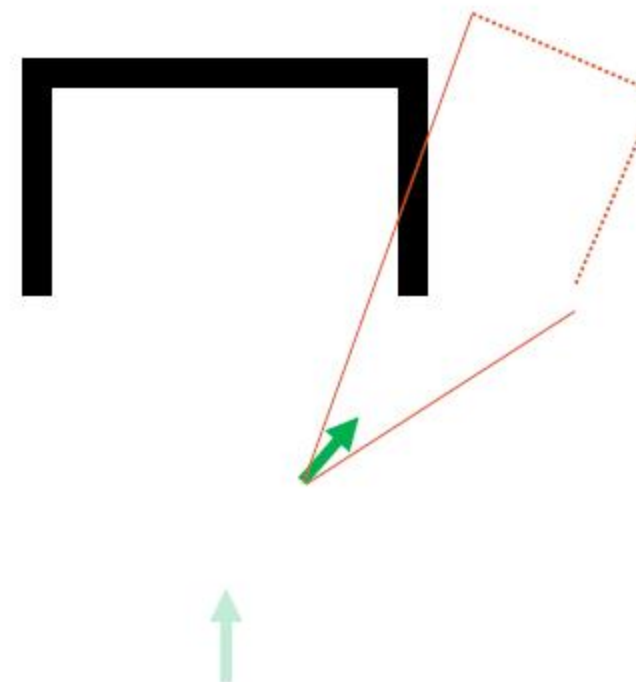
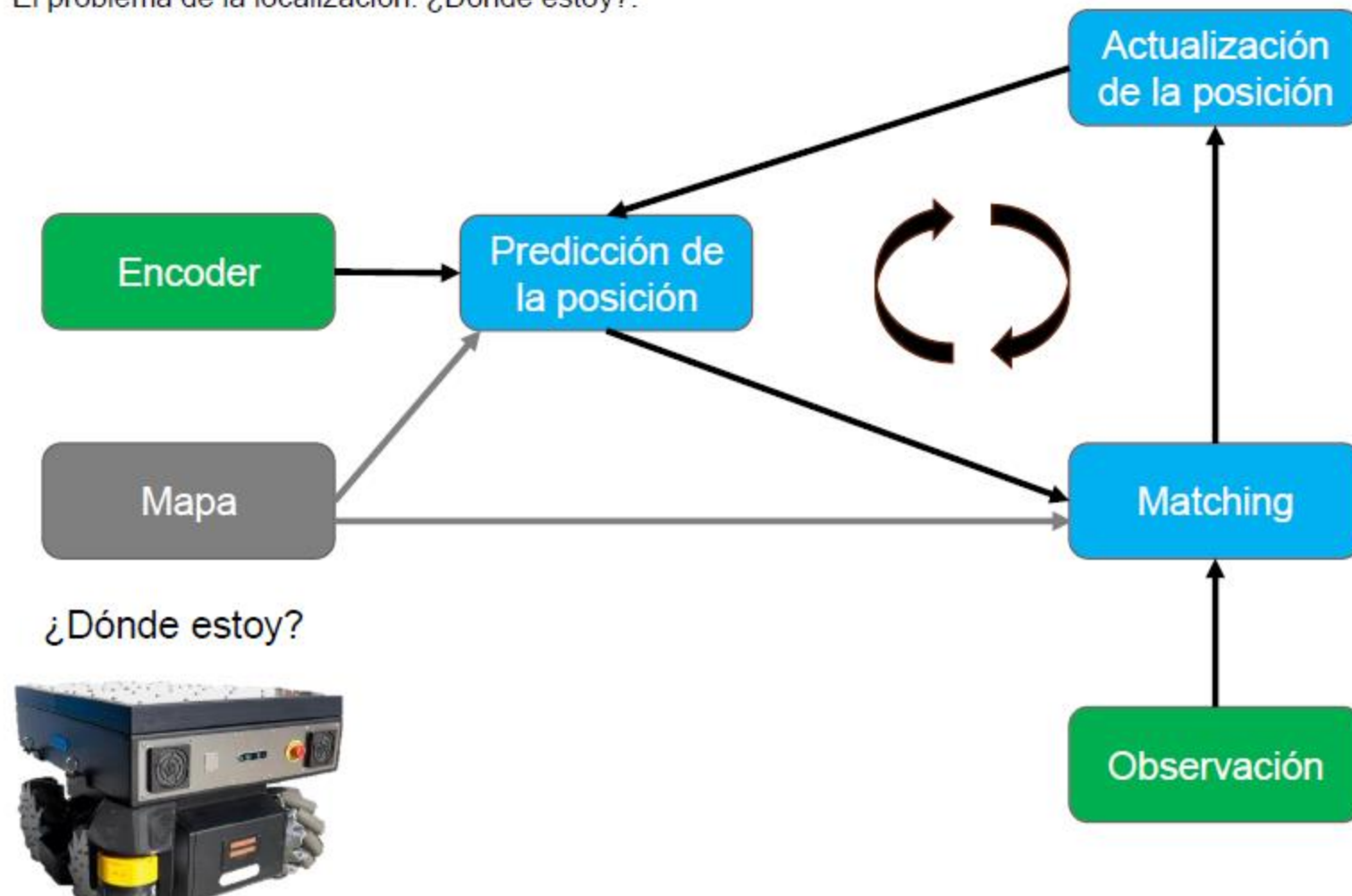
Localización

El problema de la localización: ¿Dónde estoy?.



Localización

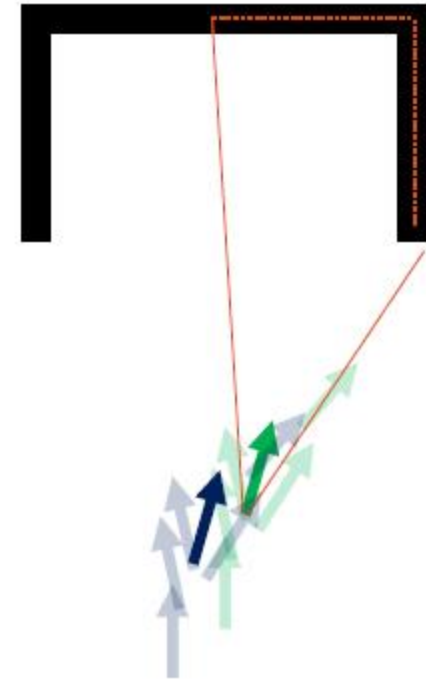
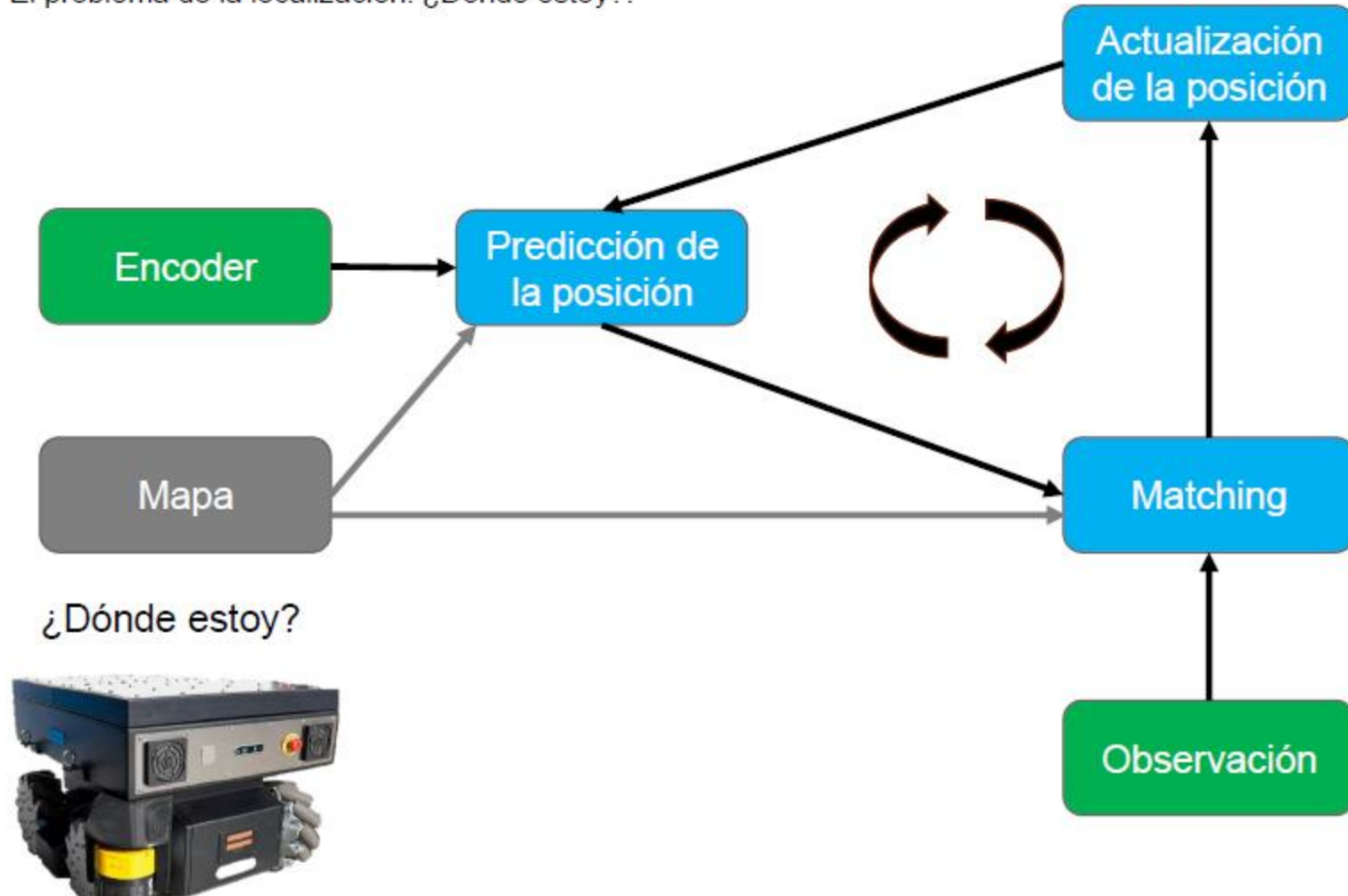
El problema de la localización: ¿Dónde estoy?



Errores en la odometria.

Localización

El problema de la localización: ¿Dónde estoy?



Ojo! El laser también tiene errores.
Ojo! Donde esta el laser?

Localización

Métodos de localización:

- **Método de localización de Montecarlo (filtro de partículas)**

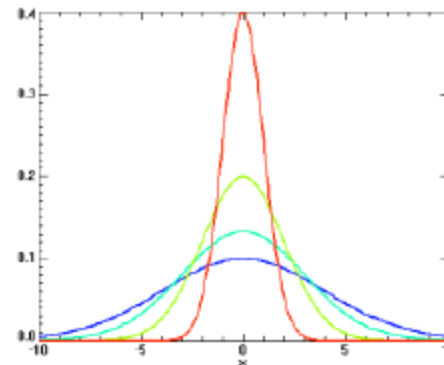
Utiliza un conjunto de partículas (muestras) para representar la distribución de probabilidad. Cada partícula representa una posible posición y orientación. Cada partícula se actualiza en base a un modelo de movimiento

Ventajas:

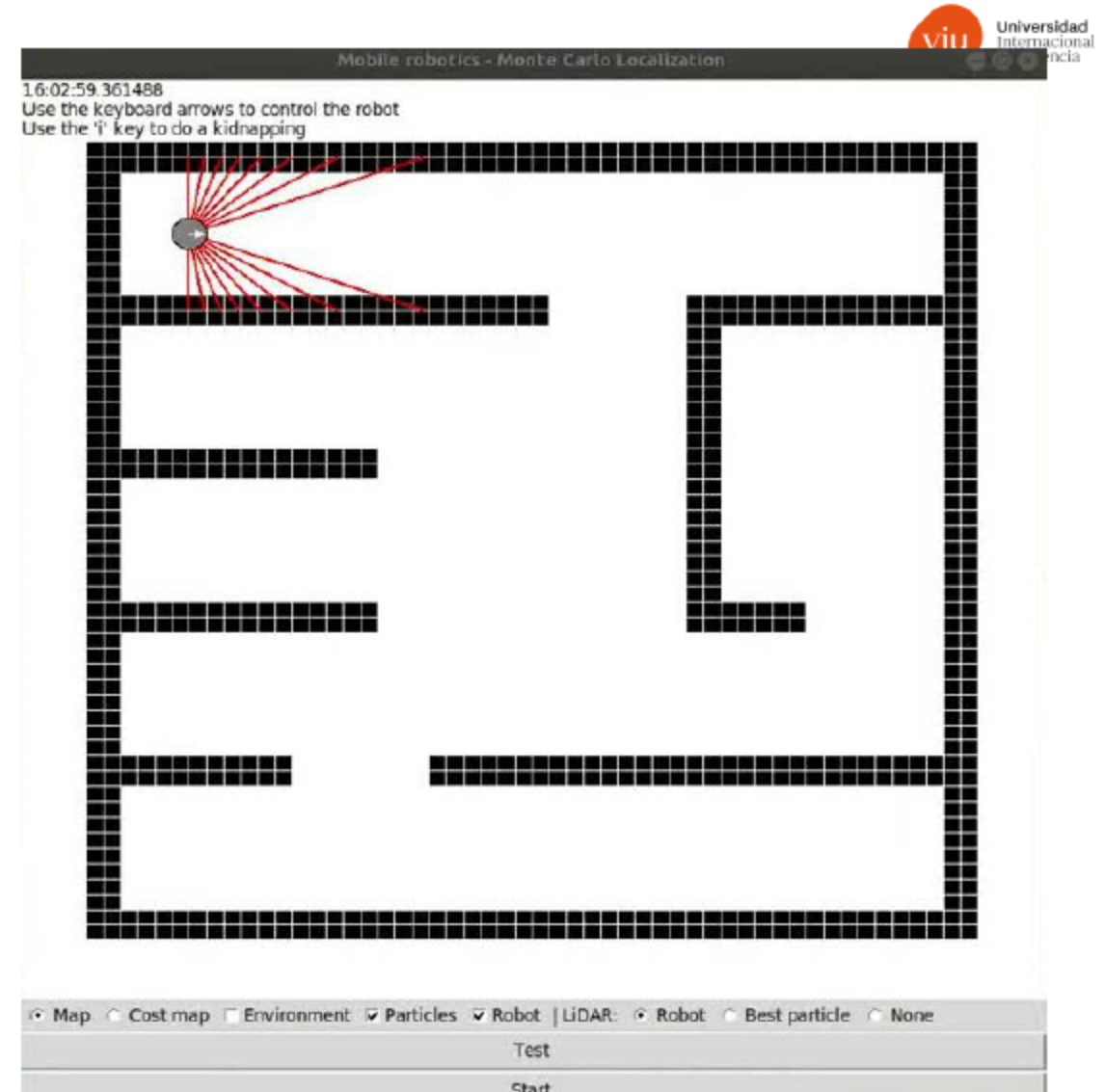
Especialmente efectivo en situaciones no lineales y donde el ruido no es gaussiano.

Robusto incluso si el robot se pierde

Muy útil en entornos desconocidos y muy cambiantes.



https://www.youtube.com/watch?v=HXVk_BERMqI



Localización

Métodos de localización:

- **Filtro de Kalman**

Algoritmo eficaz para estimar el estado interno de un sistema lineal. Óptimo cuando el ruido, las incertidumbres y las mediciones son gaussianas.

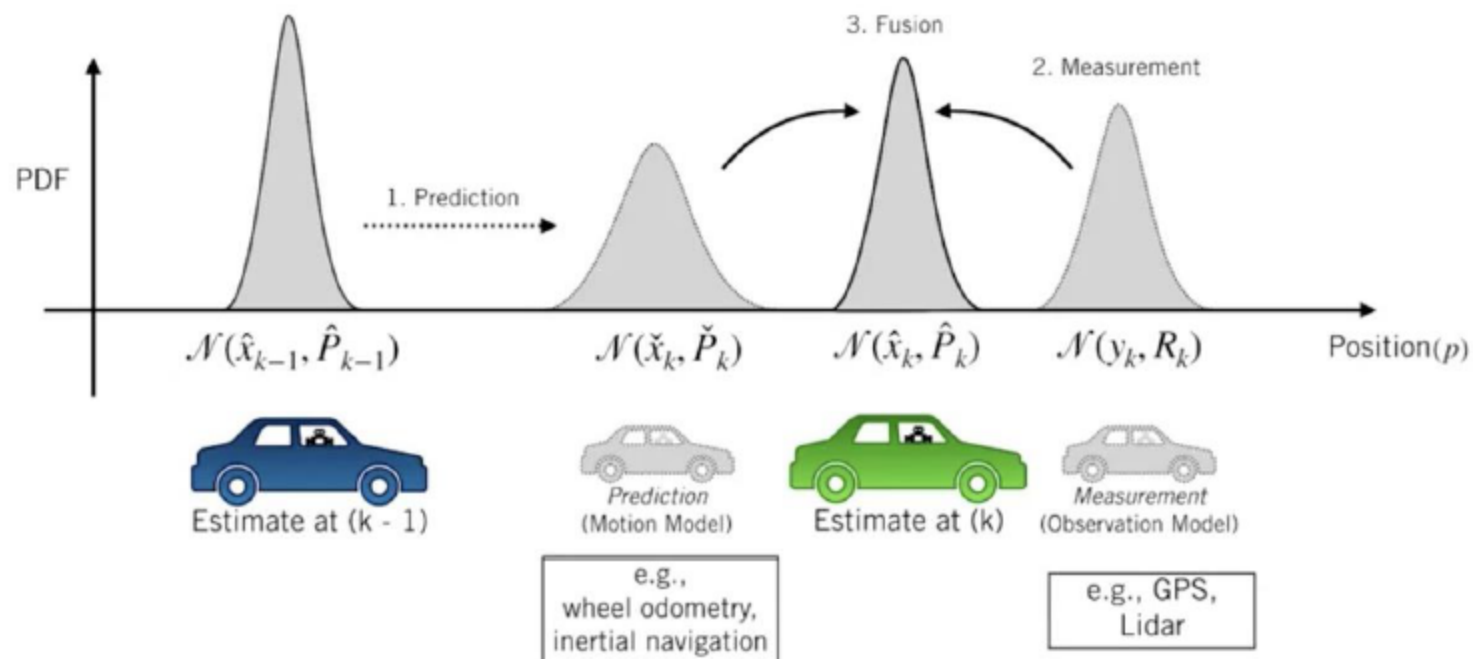
Ventajas:

Proporciona una estimación precisa si los modelos de sistema y ruido están bien definidos y son gaussianos.

Funciona bien a tiempo real

Útil para seguimiento de trayectorias.

The Kalman Filter I Prediction and Correction



Localización y mapeo

Métodos de localización:

- **SLAM (simultaneous localization and mapping)**

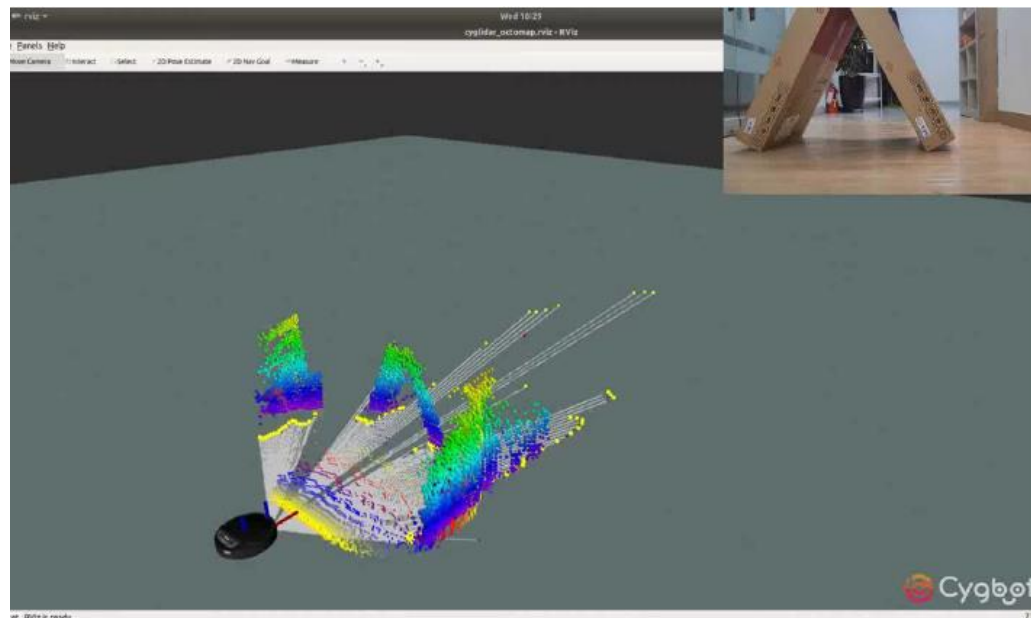
Técnica avanzada que permite a un robot construir o actualizar un mapa de un entorno desconocido mientras se ubica en el mismo. Combina elementos de localización con elementos de construcción de mapas.

Ventajas:

Puede operar sin un mapa previo.

Utiliza técnicas como fusión de sensores e integra métodos como el filtro de Kalman o el método de Montecarlo.

Esencial para empezar a trabajar en un nuevo entorno.



<https://www.youtube.com/watch?v=34n1tF5OtQU>

Collision Avoidance

Algoritmos típicos de collision avoidance:

- **Dynamic Window Approach (DWA):**

Calcula las velocidades factibles para el robot que permitan no realizar una colisión, teniendo en cuenta las velocidades y aceleraciones actuales. Maximiza una función que le permite alcanzar el objetivo gracias a una trayectoria (Requiere un modelado bueno del robot)

- **Algoritmo de campos potenciales:**

Enfoque clásico en el que se modela el objetivo como un campo atractor y todos los obstáculos como campos repulsores (Tiene problemas de mínimos locales)

- **Rapidly-exploring Random-Trees**

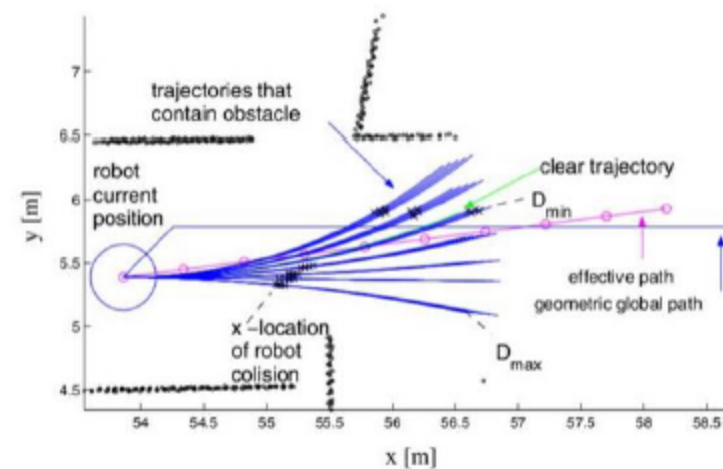
Método útil en espacios de muchas dimensiones. Extiende ramas desde el inicio hacia el objetivo de forma pseudo-aleatoria.

- **Otros:**

Vector field histogram, modelos predictivos de control, Reinforcement Learning, Deep Reinforcement Learning, Braitenberg algorithm.

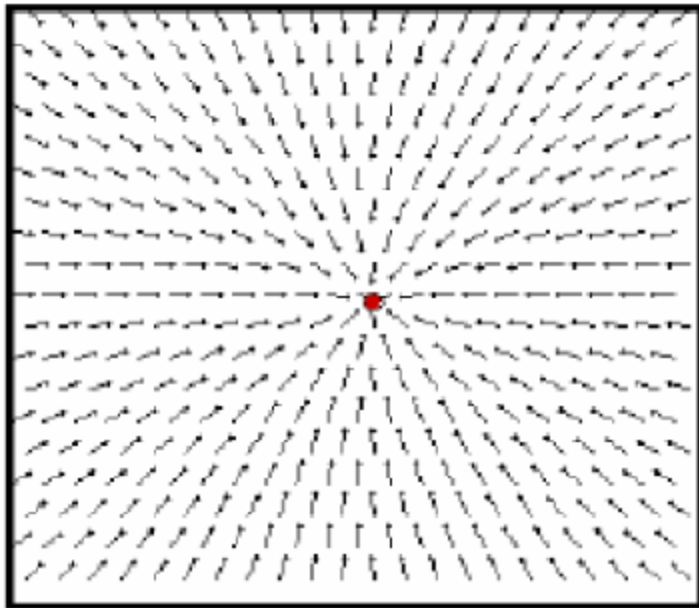
Suelen combinarse con algoritmos tipo **pure pursuit**:

Algoritmo de control simple para hacer un seguimiento de trayectorias con un robot móvil.

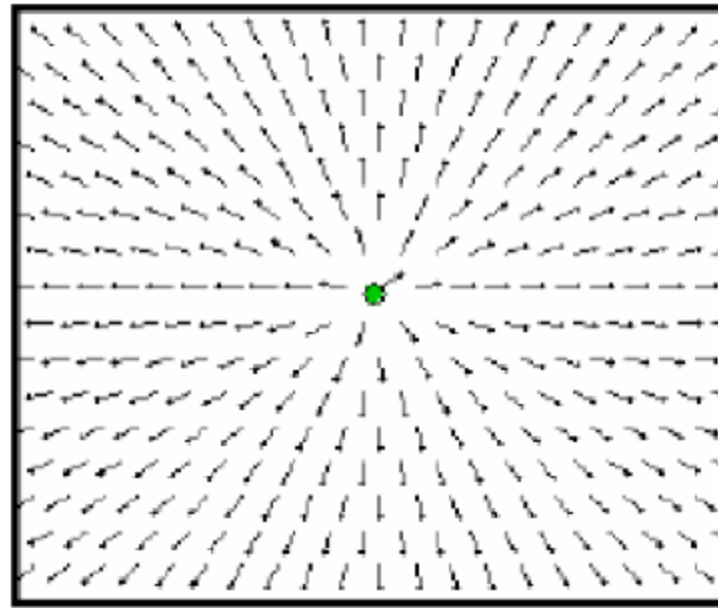


Collision Avoidance

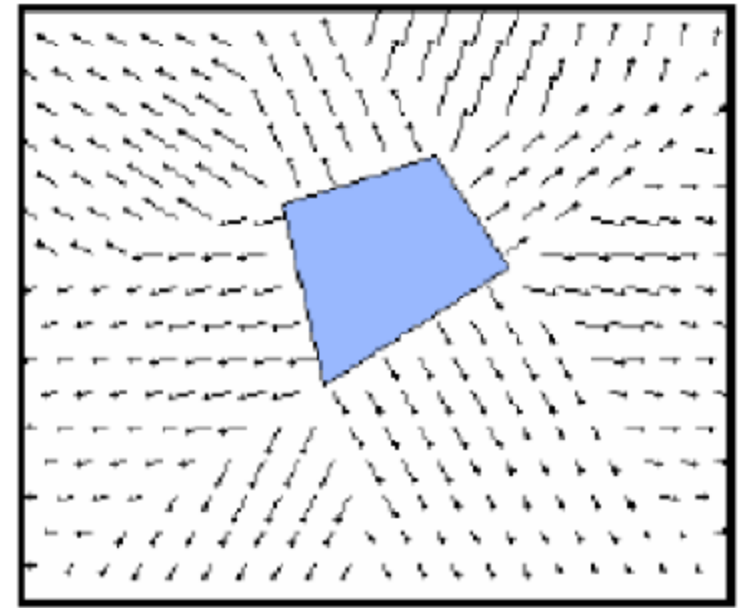
Algoritmo de campos potenciales



Atracción al objetivo (goal)



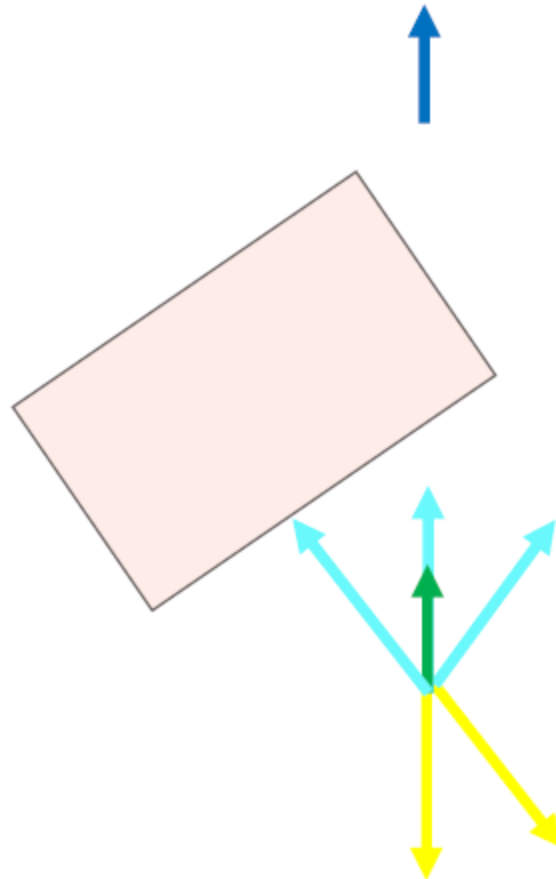
Partícula posicionada en el origen



Repulsión a los obstáculos

Collision Avoidance

Algoritmo de campos potenciales

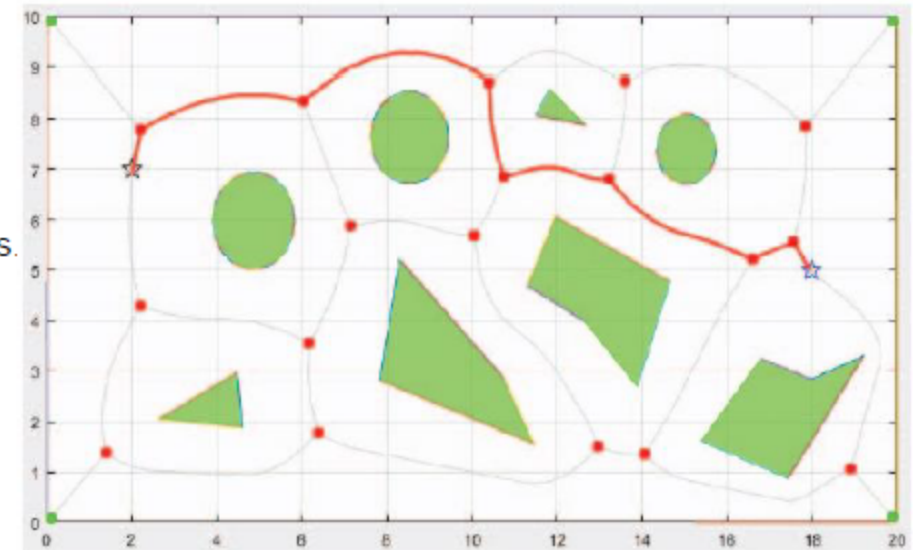


Planificación de trayectorias

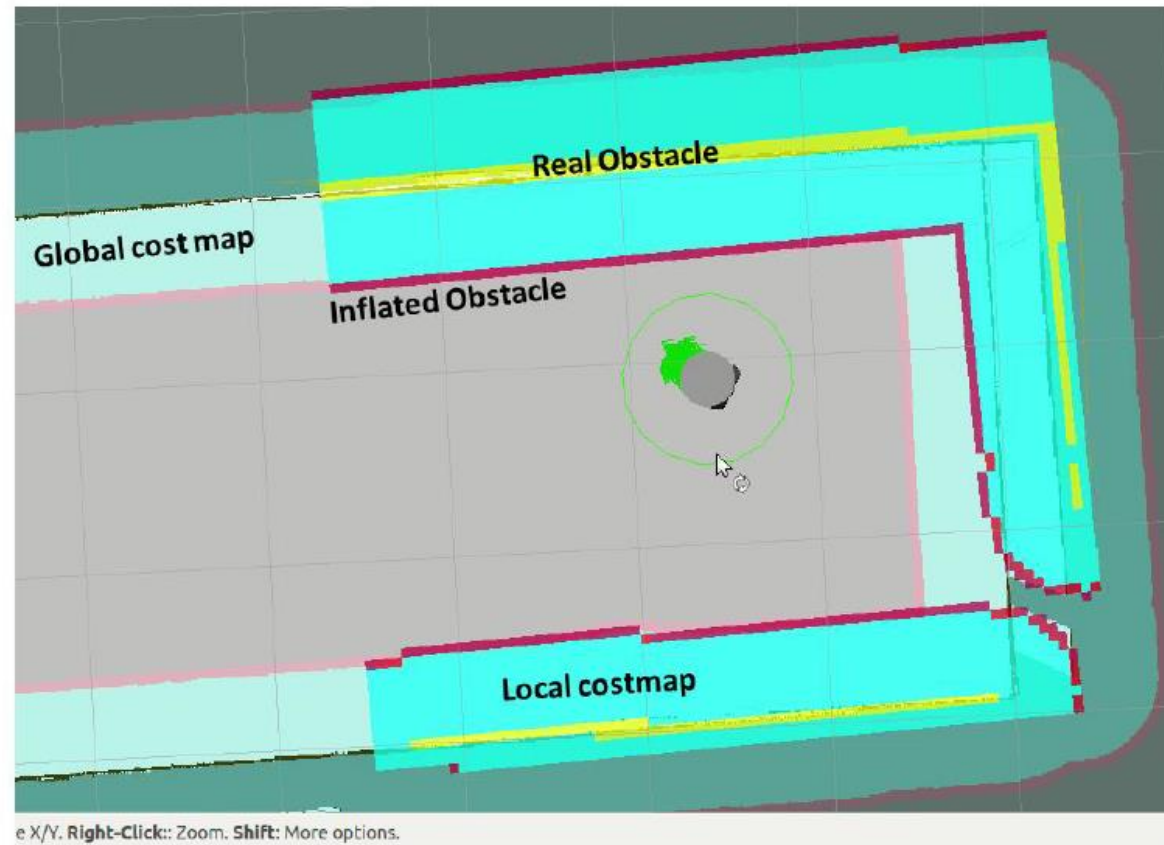
Algoritmos típicos de collision avoidance:

- **A***
Combinación de costos y trayectorias heurísticas para estimar el costo más bajo al objetivo
- **Dijkstra's algorithm**
Considerado como uno de los más simples y efectivos, parecido al A* pero con función de coste.
- **Diagramas de voronoi / cell decomposition**
Descomponen el espacio en celdas calificando áreas navegables y no navegables.
- **Otros:**
Deep Reinforcement Learning

Suelen combinarse con un algoritmo de collision avoidance que actúe más rápido que éste.



Planificación de trayectorias



<https://www.youtube.com/watch?v=kQVmo5UeWtw>

¿Dudas?